Django ORM Queries – Essential Cheat Sheet

This guide contains the most important and commonly used Django ORM queries. It covers CRUD operations, filtering, aggregation, relationships, optimization, annotations, transactions, and advanced querying techniques used in real-world Django applications.

---

1. Basic Model Setup

```
from django.db import models

class Author(models.Model):
    name = models.CharField(max_length=100)

class Book(models.Model):
    title = models.CharField(max_length=200)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    published = models.BooleanField(default=True)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
```

2. Create Records

```
# Create and save
book = Book(title="Django Mastery", price=499, author=author)
book.save()

# Shortcut create
Book.objects.create(
    title="Python Guide",
    price=299,
    author=author
)

# Bulk create
Book.objects.bulk_create([
    Book(title="Book 1", price=100, author=author),
    Book(title="Book 2", price=200, author=author),
])
```

3. Retrieve Records

```
# Get all records
Book.objects.all()

# Get single object
Book.objects.get(id=1)

# First and last
Book.objects.first()
Book.objects.last()

# Latest
Book.objects.latest('id')

# Exists
Book.objects.filter(price__gt=100).exists()

# Count
Book.objects.count()
```

4. Filtering Queries

```
# Exact match
Book.objects.filter(title="Python Guide")

# Greater than
Book.objects.filter(price__gt=200)

# Less than
Book.objects.filter(price__lt=500)

# Contains
Book.objects.filter(title__contains="Python")

# Case insensitive contains
Book.objects.filter(title__icontains="python")

# Startswith / Endswith
Book.objects.filter(title__startswith="Django")
Book.objects.filter(title__endswith="Guide")

# IN query
Book.objects.filter(id__in=[1,2,3])

# Between range
Book.objects.filter(price__range=(100, 500))

# Null checks
Book.objects.filter(author__isnull=True)
```

5. Exclude Queries

```
Book.objects.exclude(price__lt=100)

Book.objects.exclude(title__icontains="test")
```

6. Ordering

```
# Ascending
Book.objects.order_by('price')

# Descending
Book.objects.order_by('-price')

# Multiple fields
Book.objects.order_by('author', '-price')
```

7. Update Queries

```
# Update single object
book = Book.objects.get(id=1)
book.price = 999
book.save()

# Bulk update
Book.objects.filter(price__lt=100).update(price=150)
```

8. Delete Queries

```
# Delete one
book = Book.objects.get(id=1)
book.delete()

# Delete queryset
Book.objects.filter(price__lt=50).delete()
```

9. Aggregation

```
from django.db.models import Avg, Sum, Max, Min, Count

Book.objects.aggregate(Avg('price'))

Book.objects.aggregate(
    total=Sum('price'),
    max_price=Max('price'),
    min_price=Min('price'),
    total_books=Count('id')
)
```

10. Annotation

```
from django.db.models import Count

Author.objects.annotate(
    total_books=Count('book')
)
```

11. Q Objects

```
from django.db.models import Q

Book.objects.filter(
    Q(price__gt=100) | Q(title__icontains="django")
)

Book.objects.filter(
    Q(price__gt=100) & Q(published=True)
)
```

12. F Expressions

```
from django.db.models import F

# Increment all prices
Book.objects.update(price=F('price') + 50)
```

13. Related Object Queries

```
# Forward relationship
book.author

# Reverse relationship
author.book_set.all()

# Filter using relationship
Book.objects.filter(author__name="John")
```

14. select_related & prefetch_related

```
# SQL JOIN optimization
Book.objects.select_related('author')

# Many-to-many optimization
Author.objects.prefetch_related('book_set')
```

15. Values and Values List

```
Book.objects.values('id', 'title')

Book.objects.values_list('title', flat=True)
```

16. Distinct

```
Book.objects.values('author').distinct()
```

17. Pagination

```
from django.core.paginator import Paginator

books = Book.objects.all()
paginator = Paginator(books, 10)

page1 = paginator.page(1)
```

18. Transactions

```
from django.db import transaction

with transaction.atomic():
    Book.objects.create(title="Atomic Book")
```

19. Raw SQL

```
Book.objects.raw(
    "SELECT * FROM app_book WHERE price > %s",
    [100]
)
```

20. Common Advanced Queries

```
# Random records
Book.objects.order_by('?')

# Get only selected fields
Book.objects.only('title')

# Defer fields
Book.objects.defer('description')

# Union queries
qs1.union(qs2)

# Intersection
qs1.intersection(qs2)

# Difference
qs1.difference(qs2)
```

21. Performance Tips

```
1. Use select_related() for ForeignKey relationships.
2. Use prefetch_related() for ManyToMany relationships.
3. Avoid querying inside loops.
```

```
4. Use values() when full model objects are not required.
5. Use bulk_create() and bulk_update() for large datasets.
6. Use indexing on frequently searched fields.
7. Use queryset.exists() instead of count() when checking existence.
```

End of Django ORM Cheat Sheet. Practice these queries regularly to become efficient with Django backend development.